

BetterDev — Documento de concepção

Data: 22/09/2026 **Escopo:** dois projetos irmãos — um plugin de fluxo de desenvolvimento com IA e uma plataforma de visualização desse fluxo.

Este documento registra o diagnóstico que originou os projetos, as decisões de arquitetura já tomadas, o desenho resultante e a ordem de construção. É o ponto de partida; não é um plano de implementação (esses ficam em `plans/`).

1. Diagnóstico — o problema medido

Varredura dos projetos em `Desktop/Projetos` em 22/09/2026. Tamanhos reais dos arquivos que o `CLAUDE.md` manda ler na abertura de toda sessão:

Projeto	CLAUDE.md	MEMORY.md	ERRORS.md	Total	≈ tokens
atlas	6 KB	67 KB	57 KB	130 KB	~33k
smart-events	5 KB	185 KB	93 KB	283 KB	~71k
MDT	3 KB	90 KB	59 KB	152 KB	~38k
DT	4 KB	4 KB	3 KB	11 KB	~3k
Projetos/ (raiz)	—	432 KB	290 KB	722 KB	~180k

Estimativa a ~4 caracteres por token.

O `atlas/CLAUDE.md` instrui, nas primeiras linhas:

1. Read this `CLAUDE.md` completely.
 2. Read `MEMORY.md` for the current project state...
 3. Read `ERRORS.md` to avoid repeating known failures.

Por que isso é um problema

O custo em dinheiro é secundário. O problema real:

- a sessão entra na janela de contexto com 20–40% já comprometido;
- a compactação chega cedo;
- depois da compactação, o agente perde justamente o detalhe que foi caro carregar.

O fluxo atual fica mais burro mais rápido. E como os arquivos são append-only por design (`## Sessão 11/09/2026 – ...`), a degradação é monotônica: piora sozinha, todo dia.

Esse é o problema que os dois projetos atacam, cada um de um lado — o plugin pelo lado da escrita, a plataforma pelo lado da leitura humana.

2. Decisões tomadas (22/09/2026)

Questão	Decisão
Fonte da verdade	Híbrido: arquivos <code>.md</code> canônicos + índice em banco
Deploy / multiusuário	Local-first, sincronização via git
Escopo da v1	As três views: planos, memória/erros, mapa do projeto — com peso na qualidade visual
Integração com a IA	CLI + hooks
Forma dos planos	Cada plano é um fluxo , não uma lista linear

3. A espinha — um schema, três consumidores

As decisões acima convergem num único ponto de design: o plano deixa de ser prosa e vira grafo declarado.

```
---
id: e4-comparacao-espacial
title: Comparação espacial com e sem edificações
status: active          # draft | active | blocked | done
branch: feat/e4-viewshed
steps:
  - id: s1
    title: Extrair máscara de edificações do viewshed
    needs: []
    files: [src/geom/viewshed.py, src/geom/buildings.py]
    status: done
    commit: a3f2c91
  - id: s2
    title: Endpoint de diff A/B
    needs: [s1]
    files: [api/routes/analysis.py]
    status: active
  - id: s3
    title: Overlay no globo
    needs: [s2]
    status: todo
---
```

O mesmo bloco serve três consumidores sem tradução nenhuma:

1. A IA carrega só o passo ativo e os arquivos listados nele — nunca o plano inteiro, nunca o histórico.
2. A UI lê `needs:` como arestas e renderiza o fluxo de graça — ninguém desenha nada à mão.
3. O CLI/hook move `status` e carimba `commit` conforme o git avança.

`needs:` é o campo que separa isso de um Trello: dependência declarada mostra o que está *bloqueado por* o quê, e o caminho crítico sai de graça. Board mantido à mão nunca tem isso, porque ninguém mantém.

4. Projeto 1 — Plugin

4.1 Estrutura de arquivos

```
.dev/
  CLAUDE.md           ~120 linhas · estável · IMUTÁVEL durante a sessão
  state.md            ~30 linhas · gerado por hook (branch, diff, passo ativo)
  memory/
    index.md          1 linha por entrada: [slug] #tag – gancho
    2026-09-11-enquadramento-camera.md (máx 40 linhas)
  errors/
    index.md          ERR-042 | #build | gancho | aberto/resolvido
    ERR-042.md
  plans/
    active/
    done/
  map.md
  DESIGN.md
```

Abertura de sessão: `CLAUDE.md` + `state.md` + os dois `index.md` ≈ **3k tokens**, no lugar dos 33–71k atuais. Todo o resto entra por grep dirigido pelas tags do índice.

Ganho secundário, e talvez mais importante: o índice obriga cada aprendizado a caber em uma linha com tag. É um filtro de qualidade embutido.

4.2 Portabilidade entre IAs

Não existe formato que Claude Code, Cursor, Codex e Gemini CLI leiam igual:

Host	Instruções	Comandos
Claude Code	<code>CLAUDE.md</code> , skills	<code>.claude/commands/*.md</code> , plugin com <code>.claude-plugin/plugin.json</code>
Cursor	<code>.cursor/rules/*.mdc</code>	<code>.cursor/commands/*.md</code>
Codex / Copilot	<code>AGENTS.md</code>	—
Gemini CLI	<code>GEMINI.md</code>	<code>.gemini/commands/</code>

 **A verificar.** A estrutura do Claude Code é conhecida com segurança. As outras três linhas da tabela precisam ser confirmadas na documentação atual antes de virar decisão.

Abordagem: `.dev/` é neutro e canônico; um CLI `betterdev sync` gera adaptadores finos por host — dez linhas cada, no formato *“leia `.dev/commands/pLan.md` e siga”*. Toda a inteligência fica em markdown neutro. Suportar um host novo = escrever um gerador, não reescrever o plugin.

4.3 Comandos

Os comandos se partem em dois regimes econômicos opostos, e tratá-los igual é o erro que encarece os plugins concorrentes.

Geradores — rodam raro, produzem arquivo, podem gastar:

- `/setup` — modo greenfield (perguntas + estrutura do zero) e modo existente (absorve o que seria `/organize` : detecta faltantes, propõe plano de organização)
- `/plan` — emite o schema da seção 3
- `/identidade-visual` — perguntas + identidade
- `/stitch` — prompts do Stitch + `DESIGN.md`

Audidores — rodam sempre, precisam ser baratos:

- `/performance`
- `/security`

Padrão obrigatório dos auditores: **script coleta evidência, modelo só julga**. `/security` roda `npm audit`, grep de segredos e o checklist de N pontos versionado no plugin, e entrega ~200 linhas ao modelo. Deixar o modelo ler o codebase custa de 10 a 50 vezes mais e acerta menos.

Os dois comandos que faltavam e que sustentam o sistema:

- `/wrap` — fim de sessão. Escreve a entrada de memória, atualiza os índices, deduplica erro repetido com contador em vez de entrada nova, valida o orçamento de tamanho. Hoje isso é prosa dentro do `CLAUDE.md`, cumprida mal e caro. Deve rodar em modelo barato.
- `/recall <tema>` — carga dirigida de contexto. Substitui "leia o [MEMORY.md](#)".

5. Mapa de economia de tokens

Em ordem de impacto, para o caso medido na seção 1:

1. **Índice + entradas curtas no lugar do monólito.** 10–25x na abertura de sessão. Sozinho justifica o plugin.
2. **Hook `SessionStart` escrevendo `state.md`.** `git log -5`, `branch`, `diff --stat`, passo ativo do plano. Geração custa zero tokens (é script), leitura custa ~300, e elimina cinco tool calls de reorientação.
3. **Tudo determinístico vira hook, não instrução.** Cada regra em prosa no `CLAUDE.md` é paga em todo turno e executada com uma round-trip de tool call. “Rode o build ao final” é um hook `Stop`, não uma frase.
4. **Respeitar o cache de prompt.** O prefixo (`system + CLAUDE.md`) só é barato enquanto não muda; reescrevê-lo no meio da sessão invalida o cache inteiro e você repaga a 1.25x. Daí a regra: **arquivo auto-carregado é imutável durante a sessão**; escrita vai para `state.md` e `memory/`, lidos sob demanda.
5. **Disciplina de subagente.** Subagente só compensa quando o input é grande e o output é pequeno (varredura → conclusão). Quando ele precisa re-derivar contexto que a thread principal já tem, você paga duas vezes. Regra prática: só spawna se a leitura esperada passa de ~20k tokens e a resposta cabe em 1k. É o oposto dos plugins de quinze subagentes — e é o diferencial declarado deste.
6. **Contexto escopado por passo.** Garantido pelo campo `files:` do schema.
7. **Orçamento de tamanho versionado.** `index.md` ≤ 200 linhas, entrada ≤ 40 linhas, `CLAUDE.md` ≤ 150 linhas; um hook falha o `/wrap` se estourar. Sem isso, o sistema regride ao estado atual em três meses.
8. **Roteamento de modelo.** Atualização de índice, checagem de formato e changelog em modelo barato.
9. **Medição.** Hook que loga tokens por arquivo lido. Sem número, a otimização é no escuro — e é essa métrica que alimenta a view de atividade dos agentes, se ela existir.

6. Projeto 2 — Visualizador

6.1 O que ele é

Não é um Trello para desenvolvimento. **O board é uma projeção do estado do repositório.** Planos, memória, erros e git já são o banco de dados — hoje ilegíveis porque estão em arquivos de 185 KB. A plataforma é a camada de leitura humana da mesma estrutura que o plugin escreve.

Consequência de projeto: a UI define o schema. Se o board precisa mostrar status, dono e dependência, o `/plan` precisa emitir esses campos. Os dois projetos são um só contrato.

6.2 Arquitetura

```
graph TD
    A[betterdev watch → fs.watch em .dev/ → parse → SQLite local (gitignored)] --> B[UI web em localhost]
    B --> C[ação do usuário → escreve .md → git]
```

O SQLite é **cache puro, 100% reconstruível dos arquivos** (`betterdev reindex`).

Isso elimina o risco clássico do modelo híbrido: não existe reconciliação, porque o banco nunca é fonte de nada. A IA edita o arquivo direto e o board acompanha — que é exatamente o comportamento necessário.

Multiusuário acontece pelo git: cada dev roda a sua instância local apontando para o clone, e o compartilhamento é o próprio repositório. Sem auth, sem infra, sem código saindo da máquina.

6.3 As três views

Planos como fluxo. Sai de graça do campo `needs:` . Nó = passo, cor = status, aresta = dependência; clicar num nó abre o diff do `commit` . É a view que precisa estar visualmente impecável — é ela que vende o produto.

Memória e erros navegáveis. Busca e filtro por tag sobre os índices. A menos vistosa e a que resolve a dor comprovada de hoje.

Mapa do projeto. A única com risco real de virar diagrama bonito e mentiroso. Blindagem: **nós e arestas saem do código** (grafo de imports/dependências, determinístico); a IA escreve apenas o rótulo de intenção de cada módulo. A topologia nunca é inventada pelo modelo.

7. Ordem de construção

1. **Schema de plano** + `/plan` que o emite. É o contrato entre os dois projetos; tudo depende dele.
2. **Migração do atlas** . Quebrar `MEMORY.md` / `ERRORS.md` em índice + entradas, via script. Usar o projeto como cobaia e **medir o antes/depois**. Se o número não aparecer aqui, a tese está errada — e é melhor descobrir agora.
3. `/wrap` + **hooks**. Fecha o ciclo: o sistema passa a se manter sozinho.
4. `betterdev watch` + **view de fluxo**. Só depois de existirem planos reais para renderizar.

O passo 2 é o teste da hipótese inteira e pode ser feito imediatamente.

8. Questões em aberto

- Confirmar os formatos de comando/instrução de Cursor, Codex e Gemini CLI (seção 4.2).
- Definir o vocabulário fechado de tags de `memory/` e `errors/` — tag livre degrada o índice.
- Decidir se a view de atividade dos agentes (custo por sessão, arquivos tocados) entra na v1 ou espera a métrica do item 9 da seção 5 existir.
- Definir política de arquivamento: o que acontece com entradas de memória de um plano concluído há seis meses.